

```
import pandas as pd
import numpy as np
```

Obsługa **indeksowania hierarchicznego** jest ważnym elementem biblioteki pandas umożliwiającym przypisanie do jednej osi wielu poziomów indeksowania (przypisanie dwóch lub więcej indeksów).

```
data = pd.Series(np.random.uniform(size=9),
                  index=[["a", "a", "a", "b", "b", "c", "c", "d", "d"],
                        [1, 2, 3, 1, 3, 1, 2, 2, 3]])
```

data

a	1	0.082384
	2	0.090221
	3	0.970302
b	1	0.470726
	3	0.375618
c	1	0.667988
	2	0.645182
d	2	0.215328
	3	0.949657

dtype: float64

Wyświetloną czytelnie serię, której indeksem jest obiekt MultiIndex.

data.index

```
MultiIndex([( 'a', 1),
             ( 'a', 2),
             ( 'a', 3),
             ( 'b', 1),
             ( 'b', 3),
             ( 'c', 1),
             ( 'c', 2),
             ( 'd', 2),
             ( 'd', 3)],
           )
```

Obiekty o indeksie hierarchicznym obsługują tzw. **indeksowanie częściowe**. Indeksowanie to umożliwia zwięzły wybór podzbioru danych

data["b"]

1	0.470726
3	0.375618

dtype: float64

data["b":"c"]

```

b 1    0.470726
  3    0.375618
c 1    0.667988
  2    0.645182
dtype: float64

data.loc[["b", "d"]]

b 1    0.470726
  3    0.375618
d 2    0.215328
  3    0.949657
dtype: float64

```

Przykład na danych

```

# Dane sprzedażowe w dwóch miastach, w dwóch latach
df = pd.DataFrame({
    "miasto": ["Łódź", "Łódź", "Kraków", "Kraków"],
    "rok": [2023, 2024, 2023, 2024],
    "sprzedaz": [120, 145, 200, 230]
})
df

{"summary":{"\n  \"name\": \"df\",\n  \"rows\": 4,\n  \"fields\": [\n    {\n      \"column\": \"miasto\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 2,\n        \"samples\": [\n          \"Krak\u00f3w\",\n          \"\u0141\u00f3d\u017a\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\",\n        \"column\": \"rok\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": 0,\n          \"min\": 2023,\n          \"max\": 2024,\n          \"num_unique_values\": 2,\n          \"samples\": [\n            2023,\n            2024\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\",\n          \"column\": \"sprzedaz\",\n          \"properties\": {\n            \"dtype\": \"number\",\n            \"std\": 50,\n            \"min\": 120,\n            \"max\": 230,\n            \"num_unique_values\": 4,\n            \"samples\": [\n              145,\n              230\n            ],\n            \"semantic_type\": \"\",\n            \"description\": \"\"\n          }\n        ]\n      }\n    }\n  ],\n  \"type\": \"dataframe\",\n  \"variable_name\": \"df\"}

df.index

RangeIndex(start=0, stop=4, step=1)

```

Zwykły filtrowanie

Za każdym razem: filtr po mieście, filtr po roku, wybór kolumny. Trzy operacje dla jednej wartości.

```
# Sprzedaż w Łodzi w 2024 - działa, ale rozwlekłe:
df[(df.miasto == "Łódź") & (df.rok == 2024)]["sprzedaz"]

1    145
Name: sprzedaz, dtype: int64

# Sprzedaż w Krakowie w 2023 - znowu to samo:
df[(df.miasto == "Kraków") & (df.rok == 2023)]["sprzedaz"]

2    200
Name: sprzedaz, dtype: int64
```

MultIndex - indeks jako "ścieżka"

```
# Ten sam zbiór, ale z hierarchicznym indeksem:
sprzedaz = pd.Series(
    [120, 145, 200, 230],
    index=[["Łódź", "Łódź", "Kraków", "Kraków"],
           [2023, 2024, 2023, 2024]]
)
print(sprzedaz)

Łódź    2023    120
        2024    145
Kraków  2023    200
        2024    230
dtype: int64

# Sam indeks to obiekt MultiIndex:
sprzedaz.index

MultiIndex([( 'Łódź', 2023),
            ( 'Łódź', 2024),
            ('Kraków', 2023),
            ('Kraków', 2024)],
           )
```

Indeksowanie - podstawowe wzorce

```
# Pojedyncza wartość - krotka (miasto, rok):
sprzedaz["Łódź", 2024]

np.int64(145)

# Wszystko dla Łodzi (wybór zewnętrznego poziomu):
sprzedaz["Łódź"]

2023    120
2024    145
dtype: int64
```

```
# Wszystkie miasta, ale tylko rok 2023 (wewnętrzny poziom):  
sprzedaz.loc[:, 2023]
```

```
Łódź      120  
Kraków    200  
dtype: int64
```

```
# Zakres miast (uwaga: wymaga posortowanego indeksu):  
sprzedaz.sort_index().loc["Kraków":"Łódź"]
```

```
Kraków  2023    200  
         2024    230  
Łódź    2023    120  
         2024    145  
dtype: int64
```

Z Series do DataFrame i z powrotem

```
print(sprzedaz)
```

```
Łódź    2023    120  
         2024    145  
Kraków  2023    200  
         2024    230  
dtype: int64
```

```
# unstack - "wypchnięcie" wewnętrznego poziomu do kolumn:  
tabela = sprzedaz.unstack()  
print(tabela)  
print(type(tabela))
```

```
         2023  2024  
Kraków    200   230  
Łódź      120   145  
<class 'pandas.core.frame.DataFrame'>
```

```
# stack - operacja odwrotna: kolumny wracają do indeksu:  
tab_stack = tabela.stack()  
print(tab_stack)  
print(type(tab_stack))
```

```
Kraków  2023    200  
         2024    230  
Łódź    2023    120  
         2024    145  
dtype: int64  
<class 'pandas.core.series.Series'>  
  
print(df)
```

	miasto	rok	sprzedaz
0	Łódź	2023	120
1	Łódź	2024	145
2	Kraków	2023	200
3	Kraków	2024	230

```
# Konwersja płaskiej ramki na hierarchiczną - set_index:
df_hier = df.set_index(["miasto", "rok"])
print(df_hier)
print(df_hier.index)
```

		sprzedaz
miasto	rok	
Łódź	2023	120
	2024	145
Kraków	2023	200
	2024	230

```
MultiIndex([( 'Łódź', 2023),  
            ( 'Łódź', 2024),  
            ( 'Kraków', 2023),  
            ( 'Kraków', 2024)],  
            names=['miasto', 'rok'])
```

```
# I z powrotem - reset_index:
df2 = df_hier.reset_index()
print(df2)
```

	miasto	rok	sprzedaz
0	Łódź	2023	120
1	Łódź	2024	145
2	Kraków	2023	200
3	Kraków	2024	230

Tabele przestawne (ang. *Pivot Table*)

```
# Przekształcamy płaską ramkę w tabelę przestawną
df2_my_pivot = df.set_index(["miasto", "rok"])["sprzedaz"].unstack()
print(df2_my_pivot)
```

rok	2023	2024
miasto		
Kraków	200	230
Łódź	120	145

```
df_pivot = pd.pivot_table(df,  
                           index="miasto",  
                           columns="rok",  
                           values="sprzedaz")  
  
print(df_pivot)
```

rok	2023	2024
miasto		
Kraków	200.0	230.0
Łódź	120.0	145.0

```

df_kwartaly = pd.DataFrame({
    "miasto": ["Łódź"]*4 + ["Kraków"]*4,
    "rok": [2023]*4 + [2023]*4,
    "kwartał": ["Q1", "Q2", "Q3", "Q4"] * 2,
    "sprzedaz": [30, 25, 35, 30, 50, 45, 55, 50]
})

df_kwartaly_sum = pd.pivot_table(df_kwartaly,
                                  index="miasto",
                                  columns="rok",
                                  values="sprzedaz",
                                  aggfunc="sum")

print(df_kwartaly_sum)

```

rok	2023
miasto	
Kraków	200
Łódź	120

Agregacje po poziomie

```

sprzedaz.index.names = ["miasto", "rok"]
sprzedaz.groupby(level="miasto").sum()

```

miasto
Kraków
Łódź

```

dtype: int64

```

```

sprzedaz.groupby(level="rok").mean()

```

rok
2023
2024

```

dtype: float64

```

Agregacja w DataFrame z MultiIndex

```

frame = pd.DataFrame({
    "sprzedaz": [120, 145, 200, 230],
    "koszty": [80, 90, 130, 140]
}, index=[["Łódź", "Łódź", "Kraków", "Kraków"],
          [2023, 2024, 2023, 2024]])
frame.index.names = ["miasto", "rok"]
frame.groupby(level="miasto").sum()

```

```

{"summary":{"\n  \"name\": \"frame\", \n  \"rows\": 2, \n  \"fields\": [\n    {\n      \"column\": \"miasto\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 2, \n        \"samples\": [\n          \"\u0141\u00f3d\u017a\", \n          \"Krak\u00f3w\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"sprzedaz\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 116, \n        \"min\": 265, \n        \"max\": 430, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          265, \n          430 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"koszty\", \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 70, \n        \"min\": 170, \n        \"max\": 270, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          170, \n          270 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      } \n    ] \n  }, \n  \"type\": \"dataframe\"}

```

```
frame.groupby(level="miasto").agg(["sum", "mean"])
```

```

{"summary":{"\n  \"name\": \"frame\", \n  \"rows\": 2, \n  \"fields\": [\n    {\n      \"column\": \"miasto\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 2, \n        \"samples\": [\n          \"\u0141\u00f3d\u017a\", \n          \"Krak\u00f3w\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"sprzedaz\", \n      \"sum\": [\n        430 \n      ], \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 116, \n        \"min\": 265, \n        \"max\": 430, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          265, \n          430 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"sprzedaz\", \n      \"mean\": [\n        132.5 \n      ], \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 58.3630944789017, \n        \"min\": 132.5, \n        \"max\": 215.0, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          132.5, \n          215.0 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"koszty\", \n      \"sum\": [\n        270 \n      ], \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 70, \n        \"min\": 170, \n        \"max\": 270, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          170, \n          270 \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      \"column\": \"koszty\", \n      \"mean\": [\n        85.0 \n      ], \n      \"properties\": {\n        \"dtype\": \"number\", \n        \"std\": 35.35533905932738, \n        \"min\": 85.0, \n        \"max\": 135.0, \n        \"num_unique_values\": 2, \n        \"samples\": [\n          85.0, \n          135.0 \n        ], \n

```

```

{"semantic_type": "\",\n      \"description\": \"\"\n      }\n      ]\n      }\", \"type\": \"dataframe\"}

frame[\"sprzedaz\"].groupby(level=\"miasto\").agg(
    total=\"sum\", srednia=\"mean\", rozstep=lambda x: x.max()-x.min()
)

{\"summary\": \"{\\n  \\\"name\\\": \\\"    \\\"\\n  \\\"rows\\\": 2,\\n  \\\"fields\\\": [\\n    {\\n      \\\"column\\\": \\\"miasto\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"string\\\",\\n        \\\"num_unique_values\\\": 2,\\n        \\\"samples\\\": [\\n          \\\"\\u0141\\u00f3d\\u017a\\\",\\n          \\\"Krak\\u00f3w\\\"\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n      }\\n    },\\n    {\\n      \\\"column\\\": \\\"total\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"number\\\",\\n        \\\"std\\\": 116,\\n        \\\"min\\\": 265,\\n        \\\"max\\\": 430,\\n        \\\"num_unique_values\\\": 2,\\n        \\\"samples\\\": [\\n          265,\\n          430\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n      }\\n    },\\n    {\\n      \\\"column\\\": \\\"srednia\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"number\\\",\\n        \\\"std\\\": 58.33630944789017,\\n        \\\"min\\\": 132.5,\\n        \\\"max\\\": 215.0,\\n        \\\"num_unique_values\\\": 2,\\n        \\\"samples\\\": [\\n          132.5,\\n          215.0\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n      }\\n    },\\n    {\\n      \\\"column\\\": \\\"rozstep\\\",\\n      \\\"properties\\\": {\\n        \\\"dtype\\\": \\\"number\\\",\\n        \\\"std\\\": 3,\\n        \\\"min\\\": 25,\\n        \\\"max\\\": 30,\\n        \\\"num_unique_values\\\": 2,\\n        \\\"samples\\\": [\\n          25,\\n          30\\n        ],\\n        \\\"semantic_type\\\": \\\"\\\",\\n        \\\"description\\\": \\\"\\\"\\n      }\\n    }\\n  ]\\n}\", \"type\": \"dataframe\"}

```

Zadania

Sieć "ModaŁódź" ma sklepy w trzech miastach. Każdy sklep sprzedaje trzy kategorie produktów. Dane obejmują 4 kwartały 2023 i 2024 roku.

Zadanie 1 — Budowanie MultiIndex

- Utwórz ramkę `df_hi` z hierarchicznym indeksem (miasto, kategoria, rok, kwartał). Posortuj indeks.
- Wyświetl liczbę poziomów indeksu i ich nazwy.
- Ile unikalnych kombinacji indeksu istnieje? Użyj `.index` do odpowiedzi.

```

df = pd.read_csv("sklepy_moda.csv", index_col=0)
df_hi = df.set_index(["miasto", "kategoria", "rok",
"kwartał"]).sort_index()

```



```
print(df_hi.index.nlevels)
print(df_hi.index.names)
print(len(df_hi.index.unique()))

4
['miasto', 'kategoria', 'rok', 'kwartal']
72
```

Zadanie 2 — Selekcje na MultiIndex

- Wybierz wszystkie dane dla Łodzi.
- Wybierz dane dla Łodzi, kategorii Damska.
- Wybierz dane dla wszystkich miast, ale tylko rok 2024. (Wskazówka: użyj metody `xs()`)
- Wybierz Q4 z obu lat, ale tylko dla Krakowa i kategorii Męska.

```
idx = pd.IndexSlice

dane_lodz = df_hi.loc["Łódź"]
dane_lodz_damska = df_hi.loc[("Łódź", "Damska")]
dane_2024 = df_hi.xs(2024, level="rok")
dane_krakow_meska_q4 = df_hi.loc[idx["Kraków", "Męska", :, "Q4"], :]

print(dane_lodz)
print("\n---\n")
print(dane_lodz_damska)
print("\n---\n")
print(dane_2024)
print("\n---\n")
print(dane_krakow_meska_q4)
```

			przychod_tys	koszt_tys	sztuki
kategoria	rok	kwartal			
Damska	2023	Q1	50.5	35.2	615
		Q2	59.3	34.5	539
		Q3	61.9	34.3	915
		Q4	81.8	58.6	775
	2024	Q1	49.5	30.2	577
		Q2	62.5	39.8	627
		Q3	53.3	32.4	563
		Q4	85.8	55.0	1157
Dziecięca	2023	Q1	20.6	14.7	216
		Q2	31.4	19.0	370
		Q3	33.1	18.7	493
		Q4	43.6	30.7	409
	2024	Q1	35.4	24.9	301
		Q2	42.5	26.4	374
		Q3	35.7	22.0	301
		Q4			

Męska	2023	Q4	54.0	33.1	554	
		Q1	40.3	26.9	335	
		Q2	42.9	28.8	394	
		Q3	42.0	28.8	465	
	2024	Q4	63.5	36.5	728	
		Q1	47.3	29.0	550	
		Q2	47.9	31.6	445	
		Q3	52.3	29.7	490	
		Q4	71.1	39.7	730	

przychod_tys koszt_tys sztuki						
rok	kwartal					
2023	Q1	50.5	35.2	615		
	Q2	59.3	34.5	539		
	Q3	61.9	34.3	915		
	Q4	81.8	58.6	775		
2024	Q1	49.5	30.2	577		
	Q2	62.5	39.8	627		
	Q3	53.3	32.4	563		
	Q4	85.8	55.0	1157		

przychod_tys koszt_tys sztuki						
miasto	kategoria	kwartal				
Kraków	Damska	Q1	73.0	44.7	843	
		Q2	89.2	65.3	869	
		Q3	89.0	53.0	759	
		Q4	115.3	70.1	1052	
	Dziecięca	Q1	48.1	28.7	629	
		Q2	56.7	35.4	704	
		Q3	50.6	28.7	700	
		Q4	79.7	48.9	741	
	Męska	Q1	60.5	38.3	578	
		Q2	74.4	42.7	771	
		Q3	67.2	43.9	868	
		Q4	100.3	62.5	1484	
Wrocław	Damska	Q1	58.3	42.8	824	
		Q2	76.1	45.8	960	
		Q3	71.1	46.6	689	
		Q4	107.6	61.2	1536	
	Dziecięca	Q1	36.4	24.2	416	
		Q2	52.6	31.0	686	
		Q3	35.5	26.2	521	
		Q4	61.4	45.0	650	
	Męska	Q1	53.3	29.4	486	
		Q2	61.2	40.4	786	
		Q3	52.4	36.3	506	

Łódź	Damska	Q4	87.5	53.8	1157	
		Q1	49.5	30.2	577	
		Q2	62.5	39.8	627	
		Q3	53.3	32.4	563	
	Dziecięca	Q4	85.8	55.0	1157	
		Q1	35.4	24.9	301	
		Q2	42.5	26.4	374	
		Q3	35.7	22.0	301	
	Męska	Q4	54.0	33.1	554	
		Q1	47.3	29.0	550	
		Q2	47.9	31.6	445	
		Q3	52.3	29.7	490	
	Q4	71.1	39.7	730		

			przychod_tys	koszt_tys	sztuki	
miasto	kategoria	rok	kwartał			
Kraków	Męska	2023	Q4	89.4	63.8	1253
		2024	Q4	100.3	62.5	1484

Zadanie 3 — pivot_table: roczne podsumowanie

- Utwórz tabelę przestawną `tab_miasta`, która pokaże **sumę przychodów** w wierszach per miasto, w kolumnach per rok.
- Dodaj do tabeli kolumnę `zmiana_proc` — procentową zmianę przychodu między 2023 a 2024. Które miasto rośnie najszybciej?
- Utwórz drugą tabelę przestawną, w której wiersze to (miasto, kategoria), kolumny to rok, wartości to *średni przychód kwartalny*. Która kombinacja (miasto, kategoria) ma najwyższy średni przychód w 2024?

```

tab_miasta = df.pivot_table(index="miasto", columns="rok",
                             values="przychod_tys", aggfunc="sum")
tab_miasta["zmiana_proc"] = (tab_miasta[2024] - tab_miasta[2023]) /
                             tab_miasta[2023] * 100
miasto_najszybsze = tab_miasta["zmiana_proc"].idxmax()
print(miasto_najszybsze)

tab_kategorie = df.pivot_table(index=["miasto", "kategoria"],
                                columns="rok", values="przychod_tys", aggfunc="mean")
kombinacja_najwyzsza = tab_kategorie[2024].idxmax()
print(kombinacja_najwyzsza)

Łódź
('Kraków', 'Damska')
```

Zadanie 4 — Agregacja po poziomach

- a) Na `df_hi` oblicz sumę przychodów per miasto (agreguj po poziomie "miasto").
- b) Oblicz **średni przychód kwartalny per kategoria** (agreguj po poziomie "kategoria").
- c) Użyj `.agg(["sum", "mean", "max"])` na kolumnie `przychod_tys` pogrupowanej po (`miasto`, `rok`). Który wiersz ma najwyższą wartość `max`?

```
suma_miasto = df_hi.groupby(level="miasto")["przychod_tys"].sum()
srednia_kategoria = df_hi.groupby(level="kategoria")
["przychod_tys"].mean()
agregacje_miasto_rok = df_hi.groupby(level=["miasto", "rok"])
["przychod_tys"].agg(["sum", "mean", "max"])

wiersz_max = agregacje_miasto_rok["max"].idxmax()
print(wiersz_max)

('Kraków', np.int64(2023))
```

Zadanie 5 — Marża i ranking

- a) W `df_hi` dodaj kolumnę `marza_tys = przychod_tys - koszt_tys`.
- b) Utwórz tabelę przestawną: *wiersze = miasto, kolumny = kategoria, wartości = suma marży*. Które miasto jest najbardziej zyskowne? Która kategoria generuje najwyższą marżę?

```
df_hi["marza_tys"] = df_hi["przychod_tys"] - df_hi["koszt_tys"]
tab_marza = df_hi.pivot_table(index="miasto", columns="kategoria",
values="marza_tys", aggfunc="sum")

miasto_najbardziej_zyskowne = tab_marza.sum(axis=1).idxmax()
kategoria_najwyzsza_marza = tab_marza.sum(axis=0).idxmax()

print(miasto_najbardziej_zyskowne)
print(kategoria_najwyzsza_marza)

Kraków
Damska
```